```r
index=c()
for(i in 1:dim(train)[1])
{
  if(!is.null(fromJSON(train$totals[i])$transactionRevenue))
  {index = append(index,i)}
}
trainy=train[index,]
```

Figure 2. Extract response variable

```r
library(jsonlite)
library(purrr)#transpose
#jsontrans3, my newest version!
jsontrans3<-function(dat,columnindex){
  all<-lapply(dat[[columnindex]],fromJSON)
  df<-data.frame()
  for(i in 1:length(all)){
    df<-union_all(df,data.frame(purrr::transpose(all[[i]]),stringsAsFactors = FALSE))
  }
  return(df)
}
set.seed(1)
test0<-test[sample(10000),]
testdevice<-jsontrans3(test0,3)
testgeoNetwork<-jsontrans3(test0,5)
testtotals<-jsontrans3(test0,8)
testtrafficSource<-jsontrans3(test0,9)
traindevice<-jsontrans3(trainy,3)
traingeoNetwork<-jsontrans3(trainy,5)
traintotals<-jsontrans3(trainy,8)
traintrafficSource<-jsontrans3(trainy,9)
```

Figure 3. Extract features

Finally, we have the dataset with appropriate structure with response variables and all the features in a single layer without braces "{}" in between. Figure 4 and Figure 5 are some of the features after JSON processing.

```r
head(trainyall)
```

pageviews <dbl>	transactionRevenue <dbl>	newVisits <dbl>	source <chr>	isTrueDirect <lgl>	channelGrouping_com <chr>
11	37.86	1	(direct)	TRUE	Direct
10	306.67	0	google	TRUE	Organic Search
11	68.03	0	mall.googleplex.com	TRUE	Referral
12	26.25	0	mall.googleplex.com	TRUE	Referral
14	574.15	0	mall.googleplex.com	TRUE	Referral
17	8.38	0	mall.googleplex.com	TRUE	Referral

6 rows | 15-20 of 26 columns

Figure 4. The features after JSON processing

```r
head(trainyall)
```

browser_com <chr>	operatingSystem_com <chr>	deviceCategory_com <chr>	country_com <chr>	source_com <chr>	month <chr>
Chrome	Linux	desktop	United States	(direct)	9<U+6708>
Chrome	Macintosh	desktop	United States	google type	9<U+6708>
Chrome	Linux	desktop	United States	google type	9<U+6708>
Chrome	Windows, Windows Phone	desktop	United States	google type	9<U+6708>
Chrome	Macintosh	desktop	United States	google type	9<U+6708>
Chrome	Linux	desktop	United States	google type	9<U+6708>

6 rows | 21-26 of 26 columns

Figure 5. The features after JSON processing

(2) The information includes mostly descriptive words and phrases. For categorical features, we cannot label them into type numbers because numbers have quantitively information and will be misunderstood by the model. So, we applied dummy variables. For instance, we have different types of Operating Systems, like Linux, Macintosh, and Windows. We transform the single column into several separate columns, which represents "is this observation using a Linux operating System?", "is this observation using a Macintosh Operating System?" or "is this observation using a Windows Operating System?" as you can see in Figure 6. This will make the matrix more sparse but easier to interpret. Most of all, it is available for the function to process the data.

operatingSystem_comLinux <dbl>	operatingSystem_comMacintosh <dbl>
1	0
0	1
0	0
1	0
1	0
0	1

6 rows | 13-14 of 37 columns

Figure 6. The features after getting dummy

After getting dummy, the sparse matrix contains more columns with lower significance. For example, there is only one or two 1 in a column, which take up an entire feature. We combined those columns to make the data more interpretable.

(3) The main performance of this concern is missing variables. We searched for all kinds of methods dealing with the missing variables. But our missing variables are not necessarily numeric but are all categorical. It's comprehensible because the initial structure of the raw data is of the form of JSON. So, we cannot ensure the completeness of the descriptive categorical information. During the exploratory data analysis, we

- Transfer all types of unavailable data into NA, like (not available in this data set), (none) etc.
- Delete columns with NAs over 15%.
- Then for those missing categorical data, we impute them with mode values instead of other more delicate and fancy ways because there's few blank to fill.

(4) It's also necessary to check the multicollinearity for reducing the redundancy. So we do corrplot() and explore highly correlated variables, then delete variables which are less interpretive. After all the data processing, we finally get the complete dataset.

(5) Hidden information: We transformed the date into weekdays (from Sunday to Saturday) and did dummylization, which offers another significant feature.